Uber Lite Ride Sharing System

Project Proposal for UCS503P
Submitted to: Dr. Paramveer Kaur
Thapar Institute of Engineering and Technology

Ayush, CSED
Roll No: 1024030740, Email: aayush_be24@thapar.edu

Gurleen Kaur, CSED
Roll No: 1024030325, Email: gkaur6_be24@thapar.edu

Anshika, CSED
Roll No: 1024030749, Email: aanshika_be24@thapar.edu

August 13, 2026

1 Higher-order goal

. . . secondary framing

This project aims to improve the accessibility and efficiency of urban ride-sharing by reducing the time, cost, and effort required for users to find and book a suitable ride. The broader goal is to provide a lightweight and practical transportation system that connects passengers with available drivers while maintaining a simple and responsive user experience.

The immediate engineering objective is to translate the core ride-sharing workflow—from requesting a ride to matching a driver and completing the trip—into a measurable and deployable C++-based software solution. The system will focus on practical aspects such as location-based driver matching, ride booking, fare estimation, trip-status management, authentication, and reliable communication through REST APIs.

2 Time-to-value

. . . primary heuristic

- We will deliver meaningful functionality quickly by prototyping the core ride-booking workflow first: user registration, authentication, ride request, driver matching, booking confirmation, and ride-status updates.
- We will prioritize fast feedback over unnecessary complexity by implementing the system in short iterations, validating the main passenger and driver workflows early, and improving the system based on testing results.
- The first iteration will focus on a working end-to-end ride-sharing workflow rather than advanced features. This allows the project to demonstrate measurable value early while leaving room for later improvements such as optimized driver matching, improved data structures, fare adjustments, and support for multiple concurrent ride requests.
- Success will be defined by what can be delivered and measured in the first iteration: a user should be able to authenticate, request a ride, the system should identify an appropriate available driver, and the ride should progress through clearly defined states.

3 Problem Statement

Traditional transportation services can be inconvenient and difficult to manage when users have to search for available vehicles, contact drivers manually, and coordinate ride details through separate channels. Users may also face difficulties in knowing whether a driver is available, tracking the status of a ride request, and estimating the expected fare before or during a trip.

The main problems addressed by UberLite include:

- **Difficulty in finding available drivers:** Users may have to manually search for or contact drivers, leading to delays in booking a ride.
- **Lack of centralized ride management:** Ride requests, driver availability, and ride details are difficult to manage without a unified platform.
- **Limited ride status visibility:** Users may not have clear information about whether a ride is requested, accepted, in progress, or completed.
- **Uncertainty in ride information:** Users may not have a simple way to view important ride details such as pickup location, destination, driver information, and estimated fare.
- **Manual management of ride requests:** Handling bookings and updating ride information manually can result in delays, inconsistencies, and errors.

These issues create the need for a simple and centralized system that can manage the basic interaction between passengers and drivers while providing clear ride information and status updates. UberLite addresses this need through a lightweight digital platform for requesting, matching, and managing rides.

4 Proposed Solution

...Software Proposition

4.1 Overview

Build a lightweight C++-based ride-sharing backend that exposes REST APIs for passengers and drivers. The system will provide the following core functionality:

- **User registration and authentication:** Passengers and drivers can register and authenticate before accessing role-specific functionality.
- **Ride request module:** Passengers can provide pickup and destination locations and submit a ride request through the available API.
- **Driver availability and matching:** The system maintains driver availability and matches ride requests with suitable available drivers based on location.
- **Ride management workflow:** Drivers can accept or reject ride requests and update the ride status throughout the journey.
- **Fare estimation and calculation:** The system calculates an estimated and final fare using predefined pricing rules based on ride-related information.
- **Ride status tracking:** The system maintains the current status of each ride, such as Requested, Accepted, In Progress, or Completed.

- **Ride history:** Completed rides and their relevant details are stored so that previous trips can be retrieved.

The backend will communicate through REST APIs and use structured JSON data for request and response handling. A lightweight client, Postman, Swagger, or a future frontend can be used to interact with and demonstrate the APIs.

4.2 Core Workflow

1. Passenger registers and authenticates with the system.
2. Passenger submits a ride request containing pickup and destination information.
3. System validates the request and checks for available drivers.
4. System identifies a suitable driver using the driver matching mechanism.
5. Driver accepts or rejects the ride request.
6. Once accepted, the ride status changes from *Requested* to *Accepted*.
7. Driver starts the trip and the ride status changes to *In Progress*.
8. Driver completes the trip and the system calculates the final fare.
9. The completed ride is stored in the ride history.

4.3 Operational Constraints for an Educational Setting

- Keep the backend lightweight and suitable for deployment on commonly available computing resources.
- Focus on the essential ride-booking workflow rather than replicating the complete functionality of commercial ride-hailing platforms.
- Use a modular C++ architecture so that individual components can be tested and extended independently.
- Use mature technologies and libraries rather than developing infrastructure from scratch.
- Avoid unnecessary dependence on complex external services during the initial implementation.
- Prioritize correctness, reliability, efficient data management, and scalability of the core backend operations.

5 Solution Approach

. . . Engineering focus

This is an engineering course project, so the focus will be on building a reliable, modular, and scalable C++ backend rather than developing novel algorithms. The implementation will prioritize the core passenger-driver workflow, efficient driver matching, well-defined ride-status transitions, secure API access, and incremental development.

5.1 Backend Architecture

. . . C++ REST-based solution

The system will use a modular C++ backend exposed through REST APIs. The major components will be separated according to their responsibilities:

- **Authentication Module:** Handles user registration, login, authentication, and role-based access for passengers and drivers.
- **Ride Management Module:** Creates ride requests, manages ride information, and controls valid ride-status transitions.
- **Driver Management Module:** Maintains driver availability, location information, and active ride assignments.
- **Matching Module:** Searches for suitable available drivers based on their location relative to the pickup point.
- **Routing Module:** Calculates suitable routes and travel distances between relevant locations using graph-based pathfinding algorithms.
- **Fare Module:** Calculates estimated and final fares using predefined pricing rules.
- **Data Management Module:** Handles persistent storage and retrieval of users, drivers, rides, and ride history.

The C++ backend will expose these functionalities through REST endpoints, with JSON used for structured request and response data.

5.2 Driver Matching

. . . lightweight matching

Driver matching will initially use a simple location-based approach:

- Maintain the availability and current location of registered drivers.
- When a passenger requests a ride, identify currently available drivers.
- Calculate or estimate the distance between the pickup location and available drivers.
- Prioritize the nearest suitable available driver.
- Update the driver's availability after assignment so that the same driver cannot be assigned to multiple active rides.

The initial implementation will use a straightforward distance-based matching strategy. The matching module will remain independent from the rest of the system so that it can later be optimized using more efficient search or spatial data structures.

5.3 Route and Distance Calculation

. . . path-based computation

The system will use graph-based path computation where required for estimating routes and travel distance between relevant locations. Dijkstra's algorithm or A* can be used depending on the final representation of the underlying road network.

The calculated distance can be used by the fare calculation module and can also support driver-selection decisions. The routing component will remain separate from the ride-management logic so that it can be optimized independently as the system grows.

5.4 Ride Workflow Management

. . . controlled state transitions

The system will manage every ride through a clearly defined workflow:

$$\textit{Requested} \rightarrow \textit{Accepted} \rightarrow \textit{In Progress} \rightarrow \textit{Completed}$$

Only valid state transitions will be permitted. This ensures that ride information remains consistent and prevents invalid operations, such as completing a ride that has not been accepted or started.

5.5 Data Management

. . . structured and persistent data

The system will maintain structured records for passengers, drivers, and rides. Data structures will be selected according to common operations such as searching for available drivers, updating driver status, creating rides, and retrieving ride history.

PostgreSQL will be used as the relational database for persistent storage of users, drivers, rides, locations, fare details, and ride history. The database layer will be separated from the core ride-management logic so that storage-related changes do not require redesigning the complete application.

JSON serialization and deserialization will be handled using `nlohmann/json`.

5.6 Authentication and Security

. . . secure API access

Authentication will be implemented so that passengers and drivers can access only the operations relevant to their roles.

JWT-based token authentication will be used for API access, with OpenSSL providing appropriate cryptographic support where required. Input validation will be performed before processing requests to prevent invalid or inconsistent data from entering the system.

5.7 Fare Calculation

. . . deterministic pricing

The fare calculation will use a predefined pricing model rather than dynamic pricing. A simple model can be represented as:

$$Fare = BaseFare + (Distance \times Rateperkm)$$

The same calculation rules will be applied consistently so that fare estimates and final fares are predictable and easy to test.

6 Evaluation Criteria

. . . measurable and attributable

The success of UberLite will be evaluated using measurable metrics directly related to the core ride-booking workflow. The evaluation will focus on system correctness, efficiency, reliability, and backend performance rather than comparison with commercial ride-hailing platforms.

6.1 Primary Evaluation Metric

. . . ride request to driver assignment

Driver Assignment Time (DAT): The time taken between a passenger submitting a valid ride request and the system assigning an available driver.

- **Target:** The median driver assignment time should remain below a predefined threshold during testing.
- **Measurement:** The metric will be calculated using timestamps recorded when the ride request is created and when a driver is successfully assigned.
- **Attribution:** Both timestamps will be generated and stored by the backend, allowing the metric to be measured directly from system logs.

6.2 Secondary Evaluation Metrics

- **Ride Request Success Rate:** Percentage of valid ride requests that are successfully matched with an available driver.
- **Matching Accuracy:** Percentage of ride requests for which the system selects the intended nearest or most suitable available driver according to the defined matching rule.
- **Fare Calculation Accuracy:** Percentage of test rides for which the calculated fare matches the expected fare according to the predefined pricing formula.
- **Workflow Correctness:** Percentage of tested rides that successfully follow the valid sequence of states from *Requested* to *Accepted*, *In Progress*, and finally *Completed*.
- **API Response Time:** Average time taken by important backend operations such as authentication, ride creation, driver matching, and ride-status updates.
- **Reliability:** Percentage of test scenarios completed without API failures, invalid state transitions, inconsistent ride assignments, or system errors.

6.3 Pilot Validation Plan

. . . high-level

The system will be validated using controlled test scenarios that represent the main passenger and driver workflows.

- Test the system with multiple passenger and driver accounts representing realistic ride-booking scenarios.
- Measure driver assignment time for a set of ride requests under different numbers of available drivers.
- Test cases where drivers accept requests, reject requests, or become unavailable before assignment.
- Verify that invalid ride-status transitions are rejected by the system.
- Compare calculated fares against manually computed expected values using the predefined fare formula.
- Test API endpoints for valid requests, invalid input, and unauthorized access.

The collected results will be used to identify performance issues, workflow errors, and reliability problems. Improvements will then be made iteratively based on the observed results.

7 Scalability

. . . with foresight

Although UberLite is initially developed as an academic C++ backend, the system will be designed to handle growth in users, drivers, and ride requests without requiring a complete redesign.

7.1 Efficient Driver Matching

. . . reducing unnecessary search

Driver matching is the main scalability concern. Instead of repeatedly checking every driver for every ride request, appropriate C++ data structures will be used to organize available drivers and reduce unnecessary search. The matching module will remain modular so that more efficient spatial search techniques can be introduced if the number of drivers increases significantly.

7.2 Data and Backend Scalability

. . . efficient operations

User, driver, and ride information will be stored in PostgreSQL with appropriate indexing for frequently queried fields. C++ data structures will be selected to support efficient driver searches, status updates, and ride management while avoiding unnecessary full scans and data copying.

7.3 Scalability Path

. . . prototype to larger system

The initial system will target a controlled academic environment. For larger deployments, scalability can be improved through more efficient driver-search structures, optimized database queries, better handling of concurrent ride requests, and multiple backend instances when required.

8 Technology and Engine Availability

. . . mature engineering components

UberLite will use a mature set of technologies and libraries that are well suited for implementing a C++-based backend and REST API system. These components allow the project to focus on ride-booking workflow, correctness, scalability, and evaluation rather than developing basic infrastructure from scratch.

- **C++:** Used as the primary language for implementing backend logic, data structures, algorithms, and core ride-management functionality.
- **Crow/Pistache:** Used as the C++ web framework for implementing REST APIs and handling HTTP requests and responses.
- **PostgreSQL:** Used as the relational database for persistent storage of users, drivers, rides, locations, fare details, and ride history.
- **nlohmann/json:** Used for JSON serialization and deserialization in API requests and responses.
- **JWT and OpenSSL:** Used to support authentication and secure handling of user credentials and tokens.

- **CMake:** Used to manage the C++ build process and project dependencies in a consistent manner.
- **Postman:** Used to test and validate REST API endpoints during development.
- **Git/GitHub:** Used for version control, collaborative development, and maintaining the project repository.

These technologies provide a mature development environment for the project and allow the team to focus on the engineering aspects of UberLite, particularly efficient driver matching, ride workflow management, correctness, testing, and scalability.

9 Project Scope and Deliverables

The project will be developed incrementally, with the first iteration focused on implementing and demonstrating the complete core ride-booking workflow.

9.1 Initial Deliverable

. . . first iteration

- User registration and authentication for passengers and drivers.
- Ride request creation with pickup and destination information.
- Driver availability management and basic driver matching.
- Ride workflow from *Requested* to *Accepted*, *In Progress*, and *Completed*.
- Basic fare estimation and final fare calculation.
- Route and distance calculation using the selected pathfinding approach.
- Persistent storage of users, drivers, rides, and ride history.
- REST API endpoints for the core system functionality.
- Unit and integration testing of the main backend modules.

9.2 Subsequent Deliverables

. . . iteration-friendly

- Improved driver-matching and search efficiency.
- Enhanced handling of concurrent ride requests.
- Improved ride-history search and filtering.
- Additional validation, authentication, and security improvements.
- Performance testing and scalability evaluation.
- Optional lightweight client or frontend for demonstrating the backend APIs.

10 Risks and Mitigations

- **Driver matching efficiency:** A simple search may become inefficient as the number of drivers increases. This will be addressed through appropriate data structures and a modular matching component that can be optimized later.
- **Concurrent ride requests:** Multiple requests may attempt to assign the same driver. Driver availability will be verified before assignment and updated immediately after a successful assignment.
- **Data consistency:** Invalid ride states or inconsistent records may occur if operations are not validated. The system will enforce defined ride-state transitions and validate API inputs.
- **Security:** Unauthorized users may attempt to access restricted operations. JWT-based authentication and role-based access control will be applied to relevant API endpoints.
- **Evaluation ambiguity:** Performance and correctness can be difficult to assess without clearly defined metrics. Backend timestamps, test cases, and measurable evaluation criteria will be established before final testing.
- **Project complexity:** Adding too many commercial ride-hailing features may make the project difficult to complete. The initial implementation will remain focused on the core ride-booking workflow.

11 Summary

UberLite proposes a lightweight C++-based ride-sharing backend that provides the essential workflow of a ride-hailing system, including user authentication, ride requests, driver matching, route and distance calculation, fare calculation, ride-status management, and ride history.

The project is designed as an engineering-focused solution rather than a research-driven system. It emphasizes time-to-value through incremental development, measurable evaluation through driver assignment time and workflow correctness, and scalability through efficient driver matching, structured data management, and modular backend design.

The initial implementation will provide a complete and testable ride-booking workflow, while the modular architecture provides a clear path for improving performance and supporting a larger number of users, drivers, and concurrent ride requests in future iterations.